Mini-Project 6: Telemetry Dispatcher (Event-Driven Architecture)

1. Project Overview

In modern architectures (like gRPC networks or ROS2 node graphs), the system often manages a dynamic list of sensors and broadcasts their data to various listeners.

This project combines your **Polymorphic Sensor HAL** (from Mini-Project 3) with **Vectors**, **Smart Pointers**, and **Lambdas** to build a dynamic registry that polls sensors and fires inline callbacks when data is ready.

2. Core Objectives

- **Polymorphic Ownership:** Use `std::vector<std::unique_ptr<ISensor>>` to manage hardware lifetimes automatically.
- **Move Semantics:** Use `std::move` to transfer ownership of sensors into the registry.
- **Lambdas & Functional:** Use `std::function` and Lambda capture blocks `[ ]` to process sensor data locally without global variables.

3. Technical Requirements

- **Language:** C++17
- **Build System:** CMake (`06_telemetry_dispatcher`)
- **Files:** `ISensor.h` , `TempSensor.h` , `SensorRegistry.h` , `main.cpp` .

4. Functional Specifications

Part A: The Updated Interface (`ISensor.h`)

- `virtual float readValue() = 0;`
- `virtual std::string getName() const = 0;`
- `virtual ~ISensor() = default;`

(Create a quick `TempSensor` and `PressureSensor` that implement this interface, returning mock data and their respective names).

Part B: The Registry (`SensorRegistry.h`)

Create a `SensorRegistry` class.

- **Private State:** `std::vector<std::unique_ptr<ISensor>> m_sensors;`
- **Method 1:** `void addSensor(std::unique_ptr<ISensor> sensor)`
  - Takes exclusive ownership of a sensor and `push_back` s it into the vector. (Requires `std::move` inside the implementation!).
- **Method 2:** `void pollAll(std::function<void(const std::string&, float)> callback)`
  - Iterates through every sensor in `m_sensors` (using a range-based `for` loop).
  - For each sensor, calls its `getName()` and `readValue()` .

- Executes the injected `callback`, passing the name and value to it.

5. Testing & Validation (In `main.cpp`)

1. **Setup:**
   - Create an instance of `SensorRegistry`.
   - Create a `std::unique_ptr<TempSensor>` and a `std::unique_ptr<PressureSensor>`.
   - Use `std::move` to add both to the registry.
2. **The Local State:**
   - Create a local variable: `int alarm_count = 0;`
   - Create a local variable: `float threshold = 50.0f;`
3. **The Lambda Callback:**
   - Call `registry.pollAll(...)`.
   - Pass an inline Lambda as the argument. The lambda must **capture** `alarm_count` by reference `[&]` and `threshold` by value `[=]`.
   - *Inside the Lambda:* Print the sensor's name and value. If the value is `> threshold`, increment `alarm_count` and print a WARNING.
4. **Verification:**
   - Print the `alarm_count` at the very end of `main()`. It should reflect any alarms triggered inside the Lambda!